WEEK 12

Hilt & Clean Architecture

Daily study material with detailed examples

Android Development with Kotlin | Prepared for **Anjana**

As apps grow, objects need other objects: a ViewModel needs a repository, which needs an API and a database. Creating all of that by hand gets messy fast. **Dependency injection** (DI) solves it, and **Hilt** is the DI library almost every Android team uses. This week you'll also learn how teams **organise** code into layers and packages. You'll finish by refactoring the Notes app with Hilt.

Day	Topic	You will be able to...
Day 1	What is dependency injection?	Explain DI and do it by hand
Day 2	Hilt setup and @Inject	Add Hilt and inject into ViewModels
Day 3	Hilt modules	Provide Retrofit, Room and interfaces with @Provides and @Binds
Day 4	Clean architecture	Organise code into ui, domain and data layers
Day 5	Offline-first repositories	Combine network and database in one repository
Day 6	Weekly project	Refactor the Notes app with Hilt

Tip: The plain Kotlin examples (manual DI, use cases, the offline-first repository) were **run**, and the outputs are real. Hilt couldn't be downloaded here, so the Hilt code was compile-checked against stand-ins that copy its annotations and functions.

WEEK 12 • DAY 1

What Is Dependency Injection?

Today's goal: Understand dependencies, why classes shouldn't create them, and do dependency injection by hand.

1. What is a dependency?

If class A needs class B to do its job, B is a **dependency** of A. A ViewModel depends on a repository; a repository depends on an API and a DAO.

2. The problem: classes that create their own dependencies

Hard to test and hard to change

```
class NotesViewModel : ViewModel() {
    private val repository = NotesRepository(              // the ViewModel builds everything itself
        NotesDatabase.getInstance(???).noteDao()             // needs a Context: where does it come from?
    )
}
```

Problem	Why it hurts
Can't test with fake data	The ViewModel always uses the real database
Hard to change	Switching to another repository means editing every class that creates one
Duplicates	Each class might create its own database or Retrofit instance (slow, buggy)
Needs things it shouldn't know about	The ViewModel needs a Context just to build the database

3. The solution: pass dependencies in

Dependency injection simply means: a class **receives** its dependencies (usually through its constructor) instead of creating them.

Constructor injection by hand (run)

```
interface MessageSender {
    fun send(to: String, text: String): String
}

class SmsSender : MessageSender {
    override fun send(to: String, text: String) = "SMS to $to: $text"
}

class FakeSender : MessageSender {                // for tests: doesn't really send
    val sent = mutableListOf<String>()
    override fun send(to: String, text: String): String {
        sent.add(text)
        return "FAKE: $text"
    }
}

class OtpService(private val sender: MessageSender) {     // receives its dependency
    fun sendOtp(phone: String, code: Int) = sender.send(phone, "Your OTP is $code")
}

fun main() {
    val real = OtpService(SmsSender())
    println(real.sendOtp("98470 11111", 4321))

    val fake = FakeSender()
    val forTest = OtpService(fake)
    println(forTest.sendOtp("00000", 1234))
    println("Fake recorded ${fake.sent.size} message(s)")
}
```

Output

```
SMS to 98470 11111: Your OTP is 4321
FAKE: Your OTP is 1234
Fake recorded 1 message(s)
```

OtpService doesn't know **which** sender it gets. In the app it gets the real one; in a test it gets a fake. That's the power of depending on an **interface** and receiving it from outside.

4. Manual DI: an app container

Someone still has to create the objects. Without a library, you write a **container** that builds everything once:

A hand-written container (run)

```
class Database { init { println("Database created") } }
class Api { init { println("Api created") } }

class NotesRepository(private val db: Database, private val api: Api)
class SettingsRepository(private val db: Database)

class AppContainer {
    private val database by lazy { Database() }          // created once, when first needed
    private val api by lazy { Api() }

    val notesRepository by lazy { NotesRepository(database, api) }
    val settingsRepository by lazy { SettingsRepository(database) }
}

fun main() {
    val container = AppContainer()
    println("Container ready")
    container.notesRepository
    container.settingsRepository        // reuses the same Database
    println("Both repositories share one database")
}
```

Output

```
Container ready
Database created
Api created
Both repositories share one database
```

by lazy creates the object the first time it's used, then always returns the same one. This works, but in a big app the container becomes huge, and you still need ViewModel factories everywhere. **Hilt writes all of this for you.**

Practice Exercises

1. Write an interface **Clock** with fun now(): Long, a SystemClock and a FakeClock(fixedTime). Write a **Greeter(clock)** that says "Good morning" or "Good evening" from the hour, and test it with FakeClock.

2. Write an AppContainer that creates one Api and gives it to a UserRepository and a PostRepository. Print when the Api is created to prove it happens once.

3. Find a class in your team's project with @Inject constructor. List its constructor parameters: those are its dependencies.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What is a dependency?

Q2. What is dependency injection?

Q3. Why do interfaces make DI powerful?

Q4. What does by lazy do?

Q5. What problem does Hilt solve?

WEEK 12 • DAY 2

Hilt Setup and @Inject

Today's goal: Add Hilt to an app, mark the Application and Activity, and inject dependencies into classes and ViewModels.

1. Dependencies

Gradle setup (Android only, versions as in your project)

```
// build.gradle.kts (Project)
plugins {
    id("com.google.dagger.hilt.android") version "2.52" apply false
    id("com.google.devtools.ksp") version "2.0.21-1.0.28" apply false
}

// app/build.gradle.kts
plugins {
    id("com.google.dagger.hilt.android")
    id("com.google.devtools.ksp")
}
dependencies {
    implementation("com.google.dagger:hilt-android:2.52")
    ksp("com.google.dagger:hilt-compiler:2.52")
    implementation("androidx.hilt:hilt-navigation-compose:1.2.0")   // for hiltViewModel()
}
```

2. Three annotations to start

Application, Activity and an injectable class

```
@HiltAndroidApp
class MyApplication : Application()          // 1. Hilt starts here

class GreetingRepository @Inject constructor() {     // 2. Hilt knows how to create this
    fun greeting(name: String) = "Hello, $name!"
}
```

AndroidManifest.xml: register the Application class

```
<application
    android:name=".MyApplication"
    ... >
```

MainActivity (Android only)

```
@AndroidEntryPoint                              // 3. allows injection into this Activity and its screens
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent { MyAppTheme { AppNavHost() } }
    }
}
```

Annotation	Where	Meaning
@HiltAndroidApp	Your Application class (once)	Creates Hilt's container for the whole app

Annotation	Where	Meaning
@AndroidEntryPoint	Activities (and Fragments)	Hilt can inject things here and in ViewModels used here
@Inject constructor(...)	Your classes	"Hilt, this is how to create me. Give me these parameters."

Careful: Forgetting **android:name=".MyApplication"** in the manifest, or **@AndroidEntryPoint** on MainActivity, gives a crash like *"Hilt Activity must be attached to an @HiltAndroidApp Application"*.

3. Constructor injection chains

Hilt connects the whole chain

```
class Clock @Inject constructor() {
    fun hour(): Int = 9
}

class GreetingRepository @Inject constructor(private val clock: Clock) {
    fun greeting(name: String): String =
        if (clock.hour() < 12) "Good morning, $name" else "Good evening, $name"
}
```

To create GreetingRepository, Hilt sees it needs a Clock. Clock has an @Inject constructor too, so Hilt creates it first. You never write `GreetingRepository(Clock())` yourself.

4. Injecting into a ViewModel

@HiltViewModel and hiltViewModel()

```
class GreetingRepository @Inject constructor() {
    fun greeting(name: String) = "Hello, $name!"
}

@HiltViewModel
class GreetingViewModel @Inject constructor(
    private val repository: GreetingRepository
) : ViewModel() {
    private val _message = MutableStateFlow(repository.greeting("Anjana"))
    val message: StateFlow<String> = _message.asStateFlow()
}

@Composable
fun GreetingRoute(viewModel: GreetingViewModel = hiltViewModel()) {
    val message by viewModel.message.collectAsStateWithLifecycle()
    Text(message, modifier = Modifier.padding(24.dp))
}
```

Before Hilt (Week 11)	With Hilt
viewModel { NotesViewModel(NotesRepository(NotesDatabase.getInstance(context).noteDao())) }	hiltViewModel()
The screen needs a Context and knows how everything is built	The screen knows nothing about how the ViewModel is built

SavedStateHandle works too: add `private val savedStateHandle: SavedStateHandle` to an @HiltViewModel's constructor and Hilt provides it, including navigation arguments (Week 8, Day 4).

5. What Hilt can't create on its own

Hilt can create classes with an **@Inject constructor**. It can't create:

- **Interfaces** (which implementation?)
- Classes from **libraries** that you can't annotate (Retrofit, Room database, OkHttpClient)
- Objects built with a **builder** (Retrofit.Builder()...build())

For these, you write a **Hilt module**. That's tomorrow.

Practice Exercises

1. Add Hilt to your practice project: Gradle setup, @HiltAndroidApp Application (registered in the manifest) and @AndroidEntryPoint MainActivity. Build and run.

2. Create a **QuoteRepository @Inject constructor()** with a list of quotes and a random() function, and a **QuoteViewModel** that shows a random quote with a "New quote" button.

3. In your team's project, find the @HiltAndroidApp class and one @HiltViewModel. What does that ViewModel's constructor receive?

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. Where does @HiltAndroidApp go?

Q2. Why does MainActivity need @AndroidEntryPoint?

Q3. What does @Inject constructor tell Hilt?

Q4. How do you get a Hilt ViewModel in a composable?

Q5. Which things need a Hilt module?

WEEK 12 • DAY 3

Hilt Modules

Today's goal: Provide Retrofit, Room and DataStore with @Provides, bind interfaces with @Binds, and choose the right scope.

1. @Module and @Provides

A **module** is a class that tells Hilt how to create things it can't create alone. Each **@Provides** function returns one dependency.

NetworkModule

```
@Serializable data class PostDto(val id: Int, val title: String)

interface PostApi {
    @GET("posts")
    suspend fun getPosts(): List<PostDto>
}

@Module
@InstallIn(SingletonComponent::class)
object NetworkModule {

    @Provides
    @Singleton
    fun provideJson(): Json = Json { ignoreUnknownKeys = true }

    @Provides
    @Singleton
    fun provideOkHttpClient(): OkHttpClient = OkHttpClient.Builder()
        .addInterceptor(HttpLoggingInterceptor().apply { level = HttpLoggingInterceptor.Level.BASIC })
        .build()

    @Provides
    @Singleton
    fun provideRetrofit(client: OkHttpClient, json: Json): Retrofit = Retrofit.Builder()
        .baseUrl("https://jsonplaceholder.typicode.com/")
        .client(client)
        .addConverterFactory(json.asConverterFactory("application/json".toMediaType()))
        .build()

    @Provides
    @Singleton
    fun providePostApi(retrofit: Retrofit): PostApi = retrofit.create(PostApi::class.java)
}
```

Annotation	Meaning
@Module	This object contains instructions for Hilt
@InstallIn(SingletonComponent::class)	These objects live as long as the **whole app**
@Provides	"Call this function when someone needs this type"
@Singleton	Create it **once** and share it (one Retrofit, one database)
Function parameters (client, json)	Hilt provides them too, from other @Provides functions

2. A database module

DatabaseModule

```
@Entity(tableName = "notes")
data class NoteEntity(@PrimaryKey(autoGenerate = true) val id: Int = 0, val title: String)

@Dao
interface NoteDao {
    @Query("SELECT * FROM notes")
    fun observeAll(): Flow<List<NoteEntity>>
}

@Database(entities = [NoteEntity::class], version = 1, exportSchema = false)
abstract class NotesDatabase : RoomDatabase() {
    abstract fun noteDao(): NoteDao
}

@Module
@InstallIn(SingletonComponent::class)
object DatabaseModule {

    @Provides
    @Singleton
    fun provideDatabase(@ApplicationContext context: Context): NotesDatabase =
        Room.databaseBuilder(context, NotesDatabase::class.java, "notes.db").build()

    @Provides
    fun provideNoteDao(database: NotesDatabase): NoteDao = database.noteDao()
}
```

@ApplicationContext asks Hilt for the app's Context. Notice that the database **singleton code from Week 11 is gone**: @Singleton does the same job.

3. @Binds: which implementation for an interface?

Binding an interface to its implementation

```
data class Note(val id: Int, val title: String)

interface NotesRepository {                          // what the ViewModel depends on
    fun observeNotes(): Flow<List<Note>>
}

class OfflineNotesRepository @Inject constructor() : NotesRepository {   // the real one
    override fun observeNotes(): Flow<List<Note>> = flowOf(listOf(Note(1, "Hello Hilt")))
}

@Module
@InstallIn(SingletonComponent::class)
abstract class RepositoryModule {
    @Binds
    @Singleton
    abstract fun bindNotesRepository(impl: OfflineNotesRepository): NotesRepository
}

@HiltViewModel
class NotesViewModel @Inject constructor(
    repository: NotesRepository                    // gets OfflineNotesRepository
) : ViewModel() {
    val notes: StateFlow<List<Note>> =
        repository.observeNotes().stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), emptyList())
}
```

@Provides	@Binds
In an object, with a function body	In an abstract class, abstract function, no body
For library classes and builders	For "use this implementation for that interface"
You write the creation code	Hilt creates the implementation via its @Inject constructor

4. Scopes: how long objects live

Install in / scope	Lives as long as	Use for
SingletonComponent + @Singleton	The whole app	Retrofit, OkHttp, database, repositories
ViewModelComponent + @ViewModelScoped	One ViewModel	Helpers that should be shared inside one ViewModel only
No scope annotation	A new instance every time it's injected	Cheap, stateless classes (use cases, mappers)

Careful: Don't make everything @Singleton. Singletons stay in memory for the whole app's life. Use it for expensive objects and shared state (database, network, repositories), not for every small class.

5. Qualifiers: two objects of the same type

Distinguishing two dispatchers

```
@Qualifier
@Retention(AnnotationRetention.BINARY)
annotation class IoDispatcher

@Module
@InstallIn(SingletonComponent::class)
object DispatchersModule {
    @Provides
    @IoDispatcher
    fun provideIoDispatcher(): CoroutineDispatcher = Dispatchers.IO
}

class FileRepository @Inject constructor(
    @IoDispatcher private val ioDispatcher: CoroutineDispatcher
) {
    suspend fun readText(): String = withContext(ioDispatcher) { "file contents" }
}
```

Injecting dispatchers (instead of writing Dispatchers.IO directly) lets tests replace them with a test dispatcher (Week 14). Many teams do this.

Practice Exercises

1. Write a **DataStoreModule** that provides DataStore<Preferences> as a @Singleton using @ApplicationContext and the settingsDataStore extension.
2. Create **interface WeatherRepository** with an implementation **NetworkWeatherRepository @Inject constructor(api: WeatherApi)**, and bind it with @Binds.
3. Open your team's project and find the Hilt modules. Which objects are @Singleton? Is there a @Binds?

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. When do you need a @Provides function?

Q2. What does @InstallIn(SingletonComponent::class) mean?

Q3. What is the difference between @Provides and @Binds?

Q4. Why shouldn't everything be @Singleton?

Q5. What is a qualifier for?

WEEK 12 • DAY 4

Clean Architecture

Today's goal: Organise an app into ui, domain and data layers, follow the dependency rule, and know when use cases help.

1. The layers

Sketch: Google's recommended app architecture

```
+--------------------------------------------+
|  UI layer         Screens + ViewModels     |   shows state, handles user events
+--------------------------------------------+
           |  depends on
           v
+--------------------------------------------+
|  Domain layer     Use cases (OPTIONAL)     |   reusable business rules
+--------------------------------------------+
           |  depends on
           v
+--------------------------------------------+
|  Data layer       Repositories + sources   |   API, database, DataStore
+--------------------------------------------+
```

The dependency rule: arrows point **down only**. The data layer knows nothing about screens. The UI layer never talks to Retrofit or Room directly.

2. A typical package structure

Sketch: Feature-based packages, layered inside

```
com.company.app/
 |-- MyApplication.kt, MainActivity.kt
 |-- navigation/                AppNavHost.kt
 |-- core/
 |    |-- ui/                   theme, shared components
 |    |-- network/              NetworkModule.kt
 |    +-- database/             NotesDatabase.kt, DatabaseModule.kt
 +-- feature/
      |-- notes/
      |    |-- data/            NoteEntity, NoteDao, NoteDto, OfflineNotesRepository
      |    |-- domain/          Note (model), NotesRepository (interface), use cases
      |    +-- ui/              NotesScreen, NotesViewModel, NotesUiState
      +-- settings/ ...
```

Tip: Every team organises packages slightly differently. Spend your first days in a project learning **its** structure: where models, repositories and screens live. Then follow the same pattern for new code.

3. Use cases

A **use case** (also called an interactor) is a small class that does **one** business action, often combining several repositories. By convention it has a single **operator fun invoke**, so you can call it like a function.

A use case with fake repositories (run)

```
data class Product(val id: Int, val name: String, val price: Int)
data class CartItem(val productId: Int, val quantity: Int)

interface ProductRepository { fun getProduct(id: Int): Product? }
interface CartRepository { fun getItems(): List<CartItem> }

class CalculateCartTotalUseCase(
    private val products: ProductRepository,
    private val cart: CartRepository
) {
    operator fun invoke(couponPercent: Int = 0): Int {
        val subtotal = cart.getItems().sumOf { item ->
            (products.getProduct(item.productId)?.price ?: 0) * item.quantity
        }
        return subtotal - subtotal * couponPercent / 100
    }
}

fun main() {
    val products = object : ProductRepository {
        val all = listOf(Product(1, "Notebook", 45), Product(2, "Pen", 10))
        override fun getProduct(id: Int) = all.find { it.id == id }
    }
    val cart = object : CartRepository {
        override fun getItems() = listOf(CartItem(1, 2), CartItem(2, 5), CartItem(99, 1))
    }
    val calculateTotal = CalculateCartTotalUseCase(products, cart)

    println("Total: Rs ${calculateTotal()}")                // calling invoke() like a function
    println("With 10% coupon: Rs ${calculateTotal(couponPercent = 10)}")
}
```

Output

```
Total: Rs 140
With 10% coupon: Rs 126
```

object : ProductRepository { } creates a quick anonymous implementation, handy for examples and tests. Product 99 doesn't exist, so it counts as 0 instead of crashing.

Use a use case when...	Skip it when...
The same logic is needed by several ViewModels	The ViewModel just calls one repository function
The logic combines several repositories	It would only pass the call through
The rule is important and deserves its own tests	It adds a file with no real logic

4. A use case with Hilt

Injecting a use case into a ViewModel

```
data class Note(val id: Int, val title: String, val isPinned: Boolean)

interface NotesRepository { fun observeNotes(): Flow<List<Note>> }

class GetPinnedNotesUseCase @Inject constructor(private val repository: NotesRepository) {
    operator fun invoke(): Flow<List<Note>> =
        repository.observeNotes().map { notes -> notes.filter { it.isPinned } }
}

@HiltViewModel
class PinnedNotesViewModel @Inject constructor(
    getPinnedNotes: GetPinnedNotesUseCase
) : ViewModel() {
    val pinned: StateFlow<List<Note>> =
        getPinnedNotes().stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), emptyList())
}
```

5. Which model belongs where?

Model	Layer	Example
DTO	data (network)	NoteDto: matches the server JSON
Entity	data (database)	NoteEntity: matches the Room table
Domain model	domain	Note: what the app works with
UiState	ui	NotesUiState: exactly what one screen shows

Mapping functions (toNote(), toEntity()) convert between them at the layer boundaries. It may look like extra work, but it means a server or database change only touches the data layer.

Practice Exercises

1. Write a **ValidateEmailUseCase** with operator fun invoke(email: String): Boolean and test it with three emails in main().
2. Write a **GetUserGreetingUseCase** that combines a UserRepository (name) and a SettingsRepository (language: "en" or "ml") to return "Hello, Anjana" or "Namaskaram, Anjana". Use fake repositories with object : ...
3. Draw the package structure of your team's app (top-level packages only) and mark which are ui, domain and data.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What is the dependency rule?

Q2. What is a use case?

Q3. Is the domain layer required?

Q4. What are the four kinds of model in a layered app?

Q5. Why use object : Interface { } in examples?

WEEK 12 • DAY 5

Offline-First Repositories

Today's goal: Combine the network and the database in one repository so the app works offline and stays up to date.

1. The idea: the database is the single source of truth

Sketch: Offline-first data flow

```
          refresh()                          observe
   Server  ----------->  Repository  ---------->  Room  ---- Flow ---->  ViewModel  -->  Screen
          <- download -                save ->           (always shown from here)
   - Opening the app shows saved data immediately (even offline).
   - A refresh downloads new data and saves it; Room's Flow updates the screen.
```

Network only	Offline-first
Empty screen while loading, every time	Shows saved data instantly
Nothing works without internet	Old data still visible offline
Each screen fetches separately	One source of truth: the database

2. An offline-first repository, running with fakes

ArticleRepository with a fake API and DAO (run)

```
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.*

data class Article(val id: Int, val title: String)

class FakeApi(var failNext: Boolean = false) {
    private var version = 1
    suspend fun fetchArticles(): List<Article> {
        delay(100)
        if (failNext) throw java.io.IOException("No internet")
        return listOf(Article(1, "Kotlin news v$version"), Article(2, "Compose tips v$version")).also { version++ }
    }
}

class FakeArticleDao {
    private val table = MutableStateFlow<List<Article>>(emptyList())
    fun observeAll(): Flow<List<Article>> = table
    fun replaceAll(articles: List<Article>) { table.value = articles }
}

class ArticleRepository(private val api: FakeApi, private val dao: FakeArticleDao) {
    val articles: Flow<List<Article>> = dao.observeAll()          // the UI always reads the database

    suspend fun refresh(): Result<Unit> = runCatching {
        val fresh = api.fetchArticles()                           // 1. download
        dao.replaceAll(fresh)                                     // 2. save; the Flow emits automatically
    }
}

fun main() = runBlocking {
    val api = FakeApi()
    val repository = ArticleRepository(api, FakeArticleDao())

    val screen = launch { repository.articles.collect { println("Screen: ${it.map { a -> a.title }}") } }
    delay(50)

    println("Refresh 1: ${repository.refresh().isSuccess}")
```

```
    delay(50)
    api.failNext = true
    println("Refresh 2 (offline): ${repository.refresh().isSuccess}")    // fails, but old data stays on screen
    delay(50)
    api.failNext = false
    println("Refresh 3: ${repository.refresh().isSuccess}")
    delay(50)
    screen.cancel()
}
```

Output

```
Screen: []
Refresh 1: true
Screen: [Kotlin news v1, Compose tips v1]
Refresh 2 (offline): false
Refresh 3: true
Screen: [Kotlin news v2, Compose tips v2]
```

Each "Screen" line appears just after its "Refresh" line, because the collector gets its turn at the next suspension point (the delay). Now look at refresh 2: it failed (offline), but the screen didn't go blank. It kept showing the saved articles. That's the user experience offline-first gives you.

3. The ViewModel: data + refresh status

Combining saved data with a refresh state

```
data class Article(val id: Int, val title: String)

interface ArticleRepository {
    val articles: Flow<List<Article>>
    suspend fun refresh(): Result<Unit>
}

data class ArticlesUiState(
    val articles: List<Article> = emptyList(),
    val isRefreshing: Boolean = false,
    val errorMessage: String? = null
)

@HiltViewModel
class ArticlesViewModel @Inject constructor(
    private val repository: ArticleRepository
) : ViewModel() {
    private val isRefreshing = MutableStateFlow(false)
    private val errorMessage = MutableStateFlow<String?>(null)

    val uiState: StateFlow<ArticlesUiState> =
        combine(repository.articles, isRefreshing, errorMessage) { list, refreshing, error ->
            ArticlesUiState(list, refreshing, error)
        }.stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), ArticlesUiState())

    init { refresh() }

    fun refresh() {
        viewModelScope.launch {
            isRefreshing.value = true
            errorMessage.value = null
            repository.refresh().onFailure {
                errorMessage.value = "Couldn't update. Showing saved articles."
            }
            isRefreshing.value = false
        }
    }
}
```

Tip: Show errors **gently** when old data is available, for example a small banner or snackbar ("Couldn't update. Showing saved articles."), instead of a full-screen error.

4. The real data sources with Hilt

Repository implementation with Retrofit and Room

```
@Serializable data class ArticleDto(val id: Int, val title: String)

interface ArticleApi {
    @GET("articles")
    suspend fun getArticles(): List<ArticleDto>
}

@Entity(tableName = "articles")
data class ArticleEntity(@PrimaryKey val id: Int, val title: String)

@Dao
interface ArticleDao {
    @Query("SELECT * FROM articles ORDER BY id")
    fun observeAll(): Flow<List<ArticleEntity>>

    @Upsert
    suspend fun upsertAll(articles: List<ArticleEntity>)
}

data class Article(val id: Int, val title: String)

interface ArticleRepository {
    val articles: Flow<List<Article>>
    suspend fun refresh(): Result<Unit>
}

class OfflineFirstArticleRepository @Inject constructor(
    private val api: ArticleApi,
    private val dao: ArticleDao
) : ArticleRepository {

    override val articles: Flow<List<Article>> =
        dao.observeAll().map { list -> list.map { Article(it.id, it.title) } }

    override suspend fun refresh(): Result<Unit> = runCatching {
        val fresh = api.getArticles()
        dao.upsertAll(fresh.map { ArticleEntity(it.id, it.title) })
    }
}
```

Bind it with `@Binds abstract fun bind(impl: OfflineFirstArticleRepository): ArticleRepository`. The ViewModel doesn't change at all, which is a sign of good architecture.

Practice Exercises

1. Extend the fake demo: add a **lastUpdated** counter to the repository that increases on each successful refresh, and print it.

2. Change FakeApi so the second refresh returns 3 articles. Check that the screen shows 3 without any extra code.

3. Add a "Pull down to refresh" button to an Articles screen that calls viewModel.refresh(), and show a LinearProgressIndicator while isRefreshing.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What is the single source of truth in an offline-first app?

Q2. What does refresh() do?

Q3. What does the user see when a refresh fails offline?

Q4. Why didn't the ViewModel need to change when the real repository replaced the fake?

Q5. Why use runCatching in refresh()?

WEEK 12 • DAY 6

Weekly Project: Notes App with Hilt

Today's goal: Refactor the Week 11 Notes app into layers, with Hilt providing every dependency.

1. What changes

Week 11	Week 12
NotesDatabase.getInstance(context) singleton code	DatabaseModule with @Provides @Singleton
viewModel { NotesViewModel(NotesRepository(...)) }	hiltViewModel()
NotesRepository is a class	NotesRepository is an interface (domain), implemented by OfflineNotesRepository (data)
Everything in one package	data / domain / ui / di packages

2. Data, domain and DI

Data layer, domain layer and Hilt modules

```
// ---------- data/local ----------
@Entity(tableName = "notes")
data class NoteEntity(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val title: String,
    val content: String,
    @ColumnInfo(name = "is_pinned") val isPinned: Boolean = false
)

@Dao
interface NoteDao {
    @Query("SELECT * FROM notes ORDER BY is_pinned DESC, id DESC")
    fun observeAll(): Flow<List<NoteEntity>>

    @Upsert
    suspend fun upsert(note: NoteEntity)

    @Query("DELETE FROM notes WHERE id = :id")
    suspend fun deleteById(id: Int)
}

@Database(entities = [NoteEntity::class], version = 1, exportSchema = false)
abstract class NotesDatabase : RoomDatabase() {
    abstract fun noteDao(): NoteDao
}

// ---------- domain ----------
data class Note(val id: Int, val title: String, val content: String, val isPinned: Boolean)

interface NotesRepository {
    fun observeNotes(): Flow<List<Note>>
    suspend fun save(note: Note)
    suspend fun delete(id: Int)
}

// ---------- data/repository ----------
class OfflineNotesRepository @Inject constructor(private val dao: NoteDao) : NotesRepository {
    override fun observeNotes(): Flow<List<Note>> =
        dao.observeAll().map { list -> list.map { Note(it.id, it.title, it.content, it.isPinned) } }
```

```
    override suspend fun save(note: Note) =
        dao.upsert(NoteEntity(note.id, note.title.trim(), note.content.trim(), note.isPinned))

    override suspend fun delete(id: Int) = dao.deleteById(id)
}

// ---------- di ----------
@Module
@InstallIn(SingletonComponent::class)
object DatabaseModule {
    @Provides
    @Singleton
    fun provideDatabase(@ApplicationContext context: Context): NotesDatabase =
        Room.databaseBuilder(context, NotesDatabase::class.java, "notes.db").build()

    @Provides
    fun provideNoteDao(db: NotesDatabase): NoteDao = db.noteDao()
}

@Module
@InstallIn(SingletonComponent::class)
abstract class RepositoryModule {
    @Binds
    @Singleton
    abstract fun bindNotesRepository(impl: OfflineNotesRepository): NotesRepository
}
```

3. UI layer

NotesViewModel and NotesScreen

```
// ---------- ui ----------
data class NotesUiState(val notes: List<Note> = emptyList())

@HiltViewModel
class NotesViewModel @Inject constructor(
    private val repository: NotesRepository
) : ViewModel() {

    val uiState: StateFlow<NotesUiState> = repository.observeNotes()
        .map { NotesUiState(it) }
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), NotesUiState())

    fun addNote(title: String) {
        if (title.isBlank()) return
        viewModelScope.launch { repository.save(Note(0, title, "", false)) }
    }

    fun togglePin(note: Note) {
        viewModelScope.launch { repository.save(note.copy(isPinned = !note.isPinned)) }
    }

    fun delete(note: Note) {
        viewModelScope.launch { repository.delete(note.id) }
    }
}

@Composable
fun NotesRoute(viewModel: NotesViewModel = hiltViewModel()) {
    val uiState by viewModel.uiState.collectAsStateWithLifecycle()
    NotesScreen(uiState, onAdd = viewModel::addNote, onTogglePin = viewModel::togglePin, onDelete = viewModel::delete)
}

@Composable
fun NotesScreen(uiState: NotesUiState, onAdd: (String) -> Unit, onTogglePin: (Note) -> Unit, onDelete: (Note) -> Unit) {
    var draft by rememberSaveable { mutableStateOf("") }
    Column(Modifier.fillMaxSize().padding(16.dp)) {
        Row(verticalAlignment = Alignment.CenterVertically) {
```

```
            OutlinedTextField(draft, { draft = it }, label = { Text("New note") },
                singleLine = true, modifier = Modifier.weight(1f))
            IconButton(onClick = { onAdd(draft); draft = "" }) {
                Icon(Icons.Default.Add, contentDescription = "Add note")
            }
        }
        LazyColumn {
            items(uiState.notes, key = { it.id }) { note ->
                ListItem(
                    headlineContent = { Text(note.title) },
                    trailingContent = {
                        Row {
                            IconButton(onClick = { onTogglePin(note) }) {
                                Icon(if (note.isPinned) Icons.Filled.Star else Icons.Outlined.Star,
                                    contentDescription = if (note.isPinned) "Unpin" else "Pin")
                            }
                            IconButton(onClick = { onDelete(note) }) {
                                Icon(Icons.Default.Delete, contentDescription = "Delete")
                            }
                        }
                    }
                )
            }
        }
    }
}
```

4. App setup

Application and Activity (Android only)

```
@HiltAndroidApp
class NotesApplication : Application()

@AndroidEntryPoint
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent { NotesTheme { NotesRoute() } }
    }
}
// and in AndroidManifest.xml: <application android:name=".NotesApplication" ...>
```

5. Check your understanding

Question	Answer
Who creates NotesDatabase?	Hilt, via DatabaseModule.provideDatabase (once, because of @Singleton)
How does NotesViewModel get OfflineNotesRepository?	It asks for NotesRepository; RepositoryModule's @Binds says to use OfflineNotesRepository
What would you change to use a different repository?	Only the @Binds line
Does NotesScreen know about Room or Hilt?	No. It only gets NotesUiState and lambdas

6. Project challenges

- Add a **SettingsRepository** (DataStore) with Hilt, and a sort-order setting used by NotesViewModel.
- Move the pin logic into a **TogglePinUseCase** and inject it into the ViewModel.
- Add an **@IoDispatcher** qualifier and use it in a repository function that exports notes to text.
- Draw the dependency graph: which class depends on which, down to the database.

Week 12 checklist

I can...	Done?
Explain dependency injection and why it helps testing	[ ]
Do constructor injection and write a manual container	[ ]
Set up Hilt (@HiltAndroidApp, @AndroidEntryPoint)	[ ]
Use @Inject constructor, @HiltViewModel and hiltViewModel()	[ ]
Write modules with @Provides, @Binds and @Singleton	[ ]
Explain scopes and qualifiers	[ ]
Organise code into ui, domain and data layers	[ ]
Write use cases when they add value	[ ]
Build an offline-first repository	[ ]
Refactor the Notes app with Hilt	[ ]

Quick Quiz

Q1. Why is NotesRepository an interface now?

Q2. Which annotation replaces the Week 11 database singleton code?

Q3. Why does NotesRoute no longer need a Context?

Q4. Where does @Binds go?

Q5. Which layer contains NoteEntity?

Answer Key

Try every exercise yourself **before** you look here. Your solution may be different and still correct, as long as the output is right!

Day 1: What Is Dependency Injection?

Exercise 1

```
interface Clock { fun hour(): Int }
class SystemClock : Clock { override fun hour() = java.time.LocalTime.now().hour }
class FakeClock(private val fixedHour: Int) : Clock { override fun hour() = fixedHour }

class Greeter(private val clock: Clock) {
    fun greet(name: String) = if (clock.hour() < 12) "Good morning, $name" else "Good evening, $name"
}

fun main() {
    println(Greeter(FakeClock(9)).greet("Anjana"))
    println(Greeter(FakeClock(19)).greet("Anjana"))
}
```

Output

```
Good morning, Anjana
Good evening, Anjana
```

Exercise 2

```
class Api { init { println("Api created") } }
class UserRepository(val api: Api)
class PostRepository(val api: Api)

class AppContainer {
    private val api by lazy { Api() }
    val users by lazy { UserRepository(api) }
    val posts by lazy { PostRepository(api) }
}

fun main() {
    val c = AppContainer()
    println(c.users.api === c.posts.api)
}
```

Output

```
Api created
true
```

Quiz answers

Q1. An object another class needs to do its job.

Q2. Giving a class its dependencies from outside (usually through its constructor) instead of letting it create them.

Q3. You can pass any implementation, such as a real one in the app and a fake one in tests.

Q4. Creates the value the first time it's used and then reuses it.

Q5. It creates and connects all the dependencies automatically, so you don't write containers and factories.

Day 2: Hilt Setup and @Inject

Exercise 1

```
@HiltAndroidApp
class PracticeApplication : Application()
// + android:name=".PracticeApplication" in the manifest and @AndroidEntryPoint on MainActivity
```

Exercise 2

```
class QuoteRepository @Inject constructor() {
    private val quotes = listOf("Keep going.", "Small steps every day.", "Code, test, repeat.")
    fun random(): String = quotes.random()
}

@HiltViewModel
class QuoteViewModel @Inject constructor(private val repository: QuoteRepository) : ViewModel() {
    private val _quote = MutableStateFlow(repository.random())
    val quote: StateFlow<String> = _quote.asStateFlow()
    fun next() { _quote.value = repository.random() }
}

@Composable
fun QuoteRoute(viewModel: QuoteViewModel = hiltViewModel()) {
    val quote by viewModel.quote.collectAsStateWithLifecycle()
    Column(Modifier.padding(24.dp)) {
        Text(quote, style = MaterialTheme.typography.headlineSmall)
        Button(onClick = viewModel::next) { Text("New quote") }
    }
}
```

Quiz answers

Q1. On the Application class (registered in the manifest).

Q2. So Hilt can inject into it and into the ViewModels its screens use.

Q3. How to create the class, and which dependencies it needs.

Q4. hiltViewModel()

Q5. Interfaces, library classes you can't annotate, and objects built with builders.

Day 3: Hilt Modules

Exercise 1

```
val Context.settingsDataStore: DataStore<Preferences> by preferencesDataStore(name = "settings")

@Module
@InstallIn(SingletonComponent::class)
object DataStoreModule {
    @Provides
    @Singleton
    fun provideDataStore(@ApplicationContext context: Context): DataStore<Preferences> =
        context.settingsDataStore
}
```

Exercise 2

```
data class Weather(val temperature: Double)
interface WeatherApi { @GET("v1/forecast") suspend fun get(@Query("latitude") lat: Double): Weather }

interface WeatherRepository { suspend fun getWeather(lat: Double): Weather }

class NetworkWeatherRepository @Inject constructor(private val api: WeatherApi) : WeatherRepository {
    override suspend fun getWeather(lat: Double) = api.get(lat)
}

@Module
@InstallIn(SingletonComponent::class)
abstract class WeatherModule {
    @Binds
    abstract fun bindWeatherRepository(impl: NetworkWeatherRepository): WeatherRepository
}
```

Quiz answers

Q1. For types Hilt can't create through an @Inject constructor: library classes and objects built with builders.

Q2. The provided objects can live as long as the whole app.

Q3. @Provides has creation code; @Binds just maps an interface to an implementation that has an @Inject constructor.

Q4. Singletons stay in memory for the whole app's life; only expensive or shared objects need it.

Q5. Telling Hilt apart two dependencies of the same type, such as two dispatchers.

Day 4: Clean Architecture

Exercise 1

```
class ValidateEmailUseCase {
    operator fun invoke(email: String): Boolean =
        email.contains("@") && email.substringAfter("@").contains(".") && !email.contains(" ")
}

fun main() {
    val validate = ValidateEmailUseCase()
    listOf("anjana@example.com", "anjana@", "an jana@x.com").forEach { println("$it -> ${validate(it)}") }
}
```

Output

```
anjana@example.com -> true
anjana@ -> false
an jana@x.com -> false
```

Exercise 2

```
interface UserRepository { fun name(): String }
interface SettingsRepository { fun language(): String }

class GetUserGreetingUseCase(private val users: UserRepository, private val settings: SettingsRepository) {
    operator fun invoke(): String {
        val hello = if (settings.language() == "ml") "Namaskaram" else "Hello"
        return "$hello, ${users.name()}"
    }
}

fun main() {
    val users = object : UserRepository { override fun name() = "Anjana" }
    val english = object : SettingsRepository { override fun language() = "en" }
    val malayalam = object : SettingsRepository { override fun language() = "ml" }
    println(GetUserGreetingUseCase(users, english)())
    println(GetUserGreetingUseCase(users, malayalam)())
}
```

Output

```
Hello, Anjana
Namaskaram, Anjana
```

Quiz answers

Q1. Layers depend only downwards: UI to domain to data, never the other way.

Q2. A small class that performs one business action, usually with an operator fun invoke.

Q3. No, it's optional; add use cases when they bring real value.

Q4. DTO, entity, domain model and UiState.

Q5. It creates a quick anonymous implementation, useful for fakes.

Day 5: Offline-First Repositories

Exercise 1

```
import kotlinx.coroutines.*

class Repo {
    var lastUpdated = 0
        private set
    suspend fun refresh(fail: Boolean): Result<Unit> = runCatching {
        delay(50)
        if (fail) throw java.io.IOException("offline")
        lastUpdated++
    }
}

fun main() = runBlocking {
    val repo = Repo()
    repo.refresh(fail = false)
    repo.refresh(fail = true)
    repo.refresh(fail = false)
    println("Successful refreshes: ${repo.lastUpdated}")
}
```

Output

```
Successful refreshes: 2
```

Exercise 2

```
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.*

fun main() = runBlocking {
    val table = MutableStateFlow(listOf("A", "B"))
    val job = launch { table.collect { println("Screen shows ${it.size} articles") } }
    delay(50)
    table.value = listOf("A", "B", "C")          // the "second refresh" saves 3 articles
    delay(50)
    job.cancel()
}
```

Output

```
Screen shows 2 articles
Screen shows 3 articles
```

Exercise 3

```
data class ArticlesUiState(val isRefreshing: Boolean = false)

@Composable
fun RefreshBar(uiState: ArticlesUiState, onRefresh: () -> Unit) {
    Column {
        if (uiState.isRefreshing) LinearProgressIndicator(Modifier.fillMaxWidth())
        TextButton(onClick = onRefresh, enabled = !uiState.isRefreshing) { Text("Refresh") }
    }
}
```

Quiz answers

Q1. The local database.

Q2. Downloads fresh data and saves it to the database; the database Flow then updates the UI.

Q3. The previously saved data, with a gentle error message.

Q4. It depends only on the ArticleRepository interface.

Q5. To return success or failure as a Result instead of throwing.

Day 6: Weekly Project: Notes App with Hilt

Quiz answers

Q1. So the ViewModel depends on an abstraction, and the implementation (or a fake in tests) can be swapped.

Q2. @Singleton on the @Provides function.

Q3. hiltViewModel() lets Hilt build the ViewModel and all its dependencies.

Q4. In an abstract class annotated with @Module and @InstallIn.

Q5. The data layer.

Next: When you finish this week, tell your mentor and ask for the **Week 13** material. Great work, Anjana!